Opis problemu

Celem projektu było napisanie aplikacji, która nakłada na wczytany obraz cyfrowy szum o rozkładzie normalnym (Gaussa). W przeciwieństwie do zwykłego szumu losowego, gdzie każda wartość ma równą szansę wystąpienia, w rozkładzie Gaussa wartości skupiają się wokół średniej, tworząc charakterystyczny wykres "dzwonu".

Implementacja

Projekt składa się z następujących elementów:

- **GUI(C# Windows Forms)**: Obsługuje wczytanie pliku graficznego, pobranie parametrów od użytkownika (siła szumu, liczba wątków) i wyświetlenie wyniku.
- **Biblioteka DLL asemblera**: Odpowiada za generowanie i dodawanie szumu w MASM.
- **Biblioteka DLL C++**: Odpowiada za generowanie i dodawanie szumu w C++.

Algorytm generowania szumu wykorzystuje transformacje Boxa-Mullera. Metoda ta pozwala na wygenerowanie dwóch niezależnych liczb o rozkładzie normalnym(Z_0 i Z_1) na podstawie dwóch liczb o rozkładzie jednostajnym(U_1, U_2) z przedziału [0, 1]. Wzory wyglądały następująco:

$$R = \sqrt{-2\ln(U_1)}$$
$$\theta = 2\pi U_2$$
$$Z_0 = R \cdot \cos(\theta)$$
$$Z_1 = R \cdot \sin(\theta)$$

Opis parametrów wejściowych dla programu

Użytkownik komunikuje się z aplikacją za pomocą formularza Form1, definiując następujące dane wejściowe:

- Obraz wejściowy (Source Image): Użytkownik wybiera plik graficzny za pomocą okna dialogowego OpenFileDialog. Obsługiwane formaty to JPG, PNG, BMP oraz GIF. Po wczytaniu obraz jest wewnętrznie konwertowany do formatu 32bpp ARGB.
- Siła szumu (Noise Strength): Wartość liczbowa (typu double), która determinuje odchylenie standardowe w rozkładzie Gaussa. Parametr ten jest pobierany z kontrolki noiseInput. Wyższa wartość tego parametru skutkuje szerszym "dzwonem" rozkładu, co przekłada się na bardziej intensywne i widoczne zaszumienie obrazu.
- Liczba wątków (Number of threads): Liczba całkowita określająca stopień zrównoleglenia obliczeń.
- W przypadku implementacji C# + ASM: Obraz dzielony jest w pętli Parallel.For na poziome pasy (chunki), których liczba odpowiada wybranej liczbie wątków.
- W przypadku implementacji C++: Parametr ten jest przekazywany do funkcji ProcessImage, która samodzielnie tworzy wektor wątków.
- Metoda przetwarzania (Processing Method): Wybór biblioteki wykonawczej (comboBox1). Decyduje o tym, czy do obliczeń zostanie użyta procedura asemblerowa (DodajASM) wykorzystująca instrukcje wektorowe, czy funkcja w języku C++ (ProcessImage) wykorzystująca bibliotekę standardową.

Parametry techniczne procedur

Algorytmy implementujące transformację Boxa-Mullera (zarówno w ASM, jak i C++) operują na niskopoziomowych wskaźnikach i danych surowych.

Procedura Asemblerowa (DodajASM) Funkcja przyjmuje parametry wykorzystując rejestry oraz stos:

- Wskaźnik na dane obrazu (RCX): Adres początkowy (Scan0) fragmentu mapy bitowej przeznaczonego do przetworzenia przez dany wątek.
- Szerokość obrazu (RDX): Liczba pikseli w poziomie.
- Wysokość fragmentu (R8): Liczba wierszy do przetworzenia w danym wywołaniu (zależna od podziału na wątki w C#).
- Siła szumu (XMM3): Wartość zmiennoprzecinkowa używana jako mnożnik promienia w transformacji Boxa-Mullera.
- Stride ([RBP+48]): Rzeczywista szerokość wiersza w bajtach (uwzględniająca padding).

Funkcja C++ (ProcessImage) Eksportowana funkcja biblioteczna przyjmuje następujące argumenty:

- unsigned char* ptr: Wskaźnik na początek danych obrazu.
- int width / int height: Całkowite wymiary obrazu.
- double strength: Odchylenie standardowe dla generatora szumu.
- int stride: Rozmiar wiersza w bajtach, służący do obliczania przesunięć (rowPtr = basePtr + (y * stride)).
- int numThreads: Liczba wątków, na które funkcja ma podzielić pracę wewnątrz biblioteki DLL.

Opis fragmentu asemblerowego kodu źródłowego biblioteki DLL z komentarzami

Kluczowym elementem implementacji jest makro CALC_PAIR_TO_STACK oraz pętla główna wykorzystująca instrukcje wektorowe (SIMD). Poniżej przedstawiono analizę tych fragmentów.

Generowanie liczb losowych i transformacja matematyczna

Pierwszy fragment kodu (makro) odpowiada za wygenerowanie liczb pseudolosowych metodą LCG, a następnie przekształcenie ich na rozkład Gaussa przy użyciu koprocesora arytmetycznego (FPU). Wykorzystano tu instrukcje trygonometryczne i logarytmiczne operujące na stosie koprocesora.

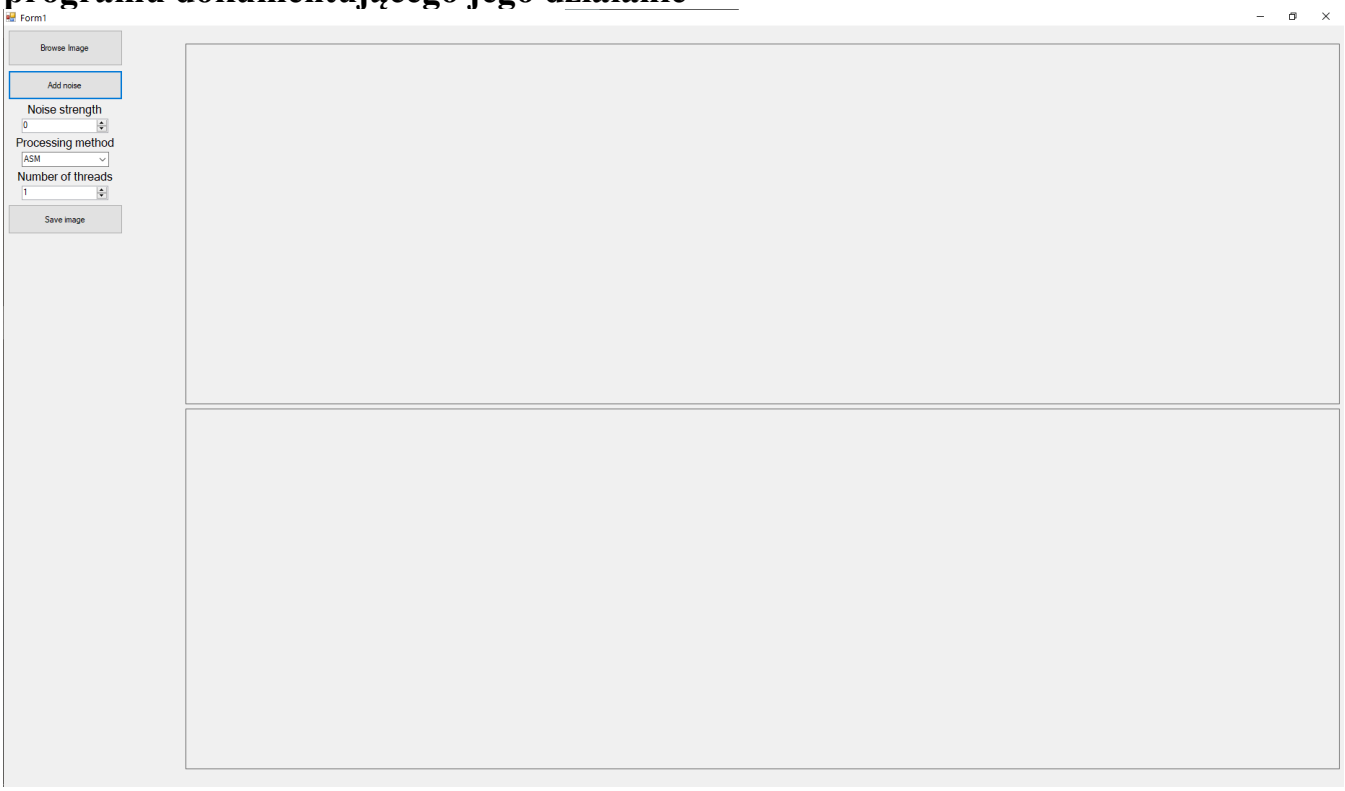
```
; Makro generujące parę liczb o rozkładzie normalnym
CALC_PAIR_TO_STACK MACRO
    ; --- Generowanie U1 (LCG) ---
    imul r11d, r11d, 1103515245 ; Mnożenie ziarna przez stałą LCG
    add r11d, 12345             ; Dodanie stałej przyrostu
    mov eax, r11d               ; Przeniesienie wyniku do akumulatora
    shr eax, 1                  ; Przesunięcie bitowe (odrzucenie najmłodszego bitu)
    or eax, 1                   ; Zapewnienie, że liczba nie jest zerem (dla logarytmu)
    mov dword ptr [rsp+8], eax  ; Zapisanie tymczasowe na stosie
    fild dword ptr [rsp+8]      ; Załadowanie liczby całkowitej na stos FPU
    fmul const_inv_max          ; Normalizacja do zakresu [0.0, 1.0] (ST(0) = u1)

    ; --- Obliczenia Transformacji Boxa-Mullera ---
    ; Wzory: R = sqrt(-2*ln(u1)), Theta = 2*pi*u2

    ; ... (Generowanie U2 analogicznie do U1) ...

    fmul const_2_pi             ; ST(0) = 2 * PI * u2 (Kąt Theta)
    fxch st(1)                  ; Zamiana wierzchołka stosu: ST(0)=u1, ST(1)=Theta
    fldln2                      ; Załadowanie ln(2)
    fxch st(1)                  ; Przygotowanie do logarytmu: ST(0)=u1
    fyl2x                       ; Obliczenie: ST(0) = ln(u1)
    fmul const_minus_2          ; Mnożenie przez -2
    fsqrt                       ; Pierwiastek: ST(0) = R (Promień)
    fmul real8 ptr [rsp]        ; Pomnożenie promienia przez siłę szumu (Strength)

    fxch st(1)                  ; ST(0)=Theta, ST(1)=R
    fsincos                     ; Jednoczesne obliczenie Sin i Cos kąta Theta

    fmul st(0), st(2)           ; Wynik 1: Cos * R
    fistp dword ptr [rsp+12]    ; Zapisanie pierwszej liczby całkowitej (szum 1)

    fmulp st(1), st(0)          ; Wynik 2: Sin * R
    fistp dword ptr [rsp+8]     ; Zapisanie drugiej liczby całkowitej (szum 2)
ENDM
```

2. Wektorowe dodawanie szumu do pikseli

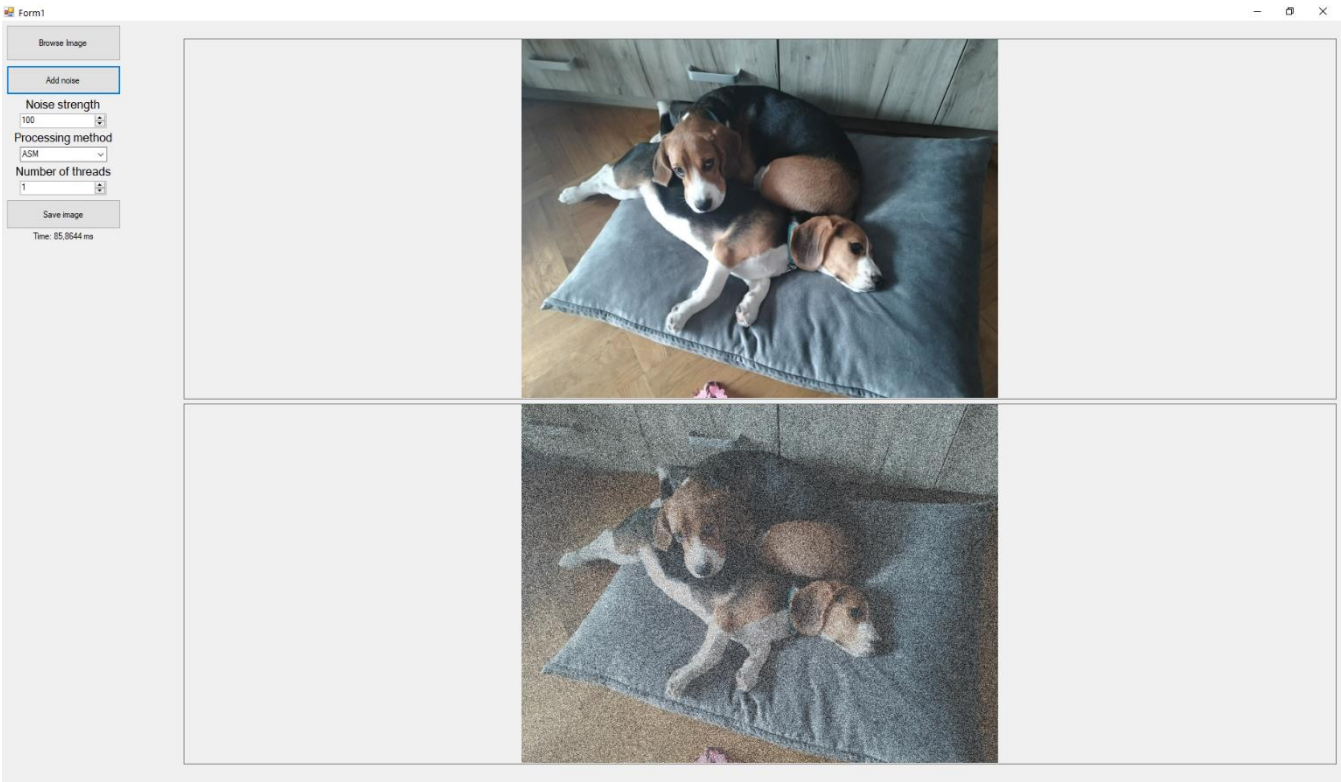
Główna pętla procedury DodajASM przetwarza obraz parami pikseli. Wykorzystano rejestry XMM oraz instrukcje z rozszerzenia SSE/AVX do równoległych operacji na danych. Kluczowym mechanizmem jest tu arytmetyka z nasyceniem, która zapobiega przekręceniu licznika po dodaniu szumu.

```
LoopX_Pairs:
    ; Sprawdzenie warunku pętli poziomej
    cmp r14, rsi
    jge CheckLastPixel          ; Skok do obsługi brzegowej, jeśli koniec wiersza

    CALC_PAIR_TO_STACK          ; Wywołanie makra (obliczenie dwóch wartości szumu)

    ; Przygotowanie wektora szumu
    vmovd xmm0, dword ptr [rsp+8]  ; Pobranie wartości szumu S1
    vmovd xmm1, dword ptr [rsp+12] ; Pobranie wartości szumu S2
    vpbroadcastw xmm0, xmm0        ; Powielenie S1 na cały rejestr XMM
    vpbroadcastw xmm1, xmm1        ; Powielenie S2 na cały rejestr XMM
    vpunpcklqdq xmm0, xmm0, xmm1   ; Połączenie wektorów szumu w jeden rejestr

    vpand xmm0, xmm0, xmmword ptr [alpha_mask_x2] ; Zerowanie szumu na pozycji Alpha

    ; Pobranie i modyfikacja pikseli
    vpmovzxbw xmm2, qword ptr [r13] ; Pobranie 2 pikseli (8 bajtów) i rozszerzenie do 16
bitów
    vpaddw xmm2, xmm2, xmm0         ; Wektorowe dodanie szumu do składowych koloru
    vpackuswb xmm2, xmm2, xmm2      ; Pakowanie do 8 bitów z nasyceniem (clamp 0-255)

    vmovq qword ptr [r13], xmm2     ; Zapisanie przetworzonych pikseli do pamięci

    add r13, 8  ; Przesunięcie wskaźnika obrazu o 8 bajtów (2 piksele * 4 kanały)
    add r14, 2  ; Inkrementacja licznika X o 2
    jmp LoopX_Pairs
```

Zrzuty ekranu wyglądu Interfejsu Użytkownika przed i w trakcie działania programu dokumentującego jego działanie



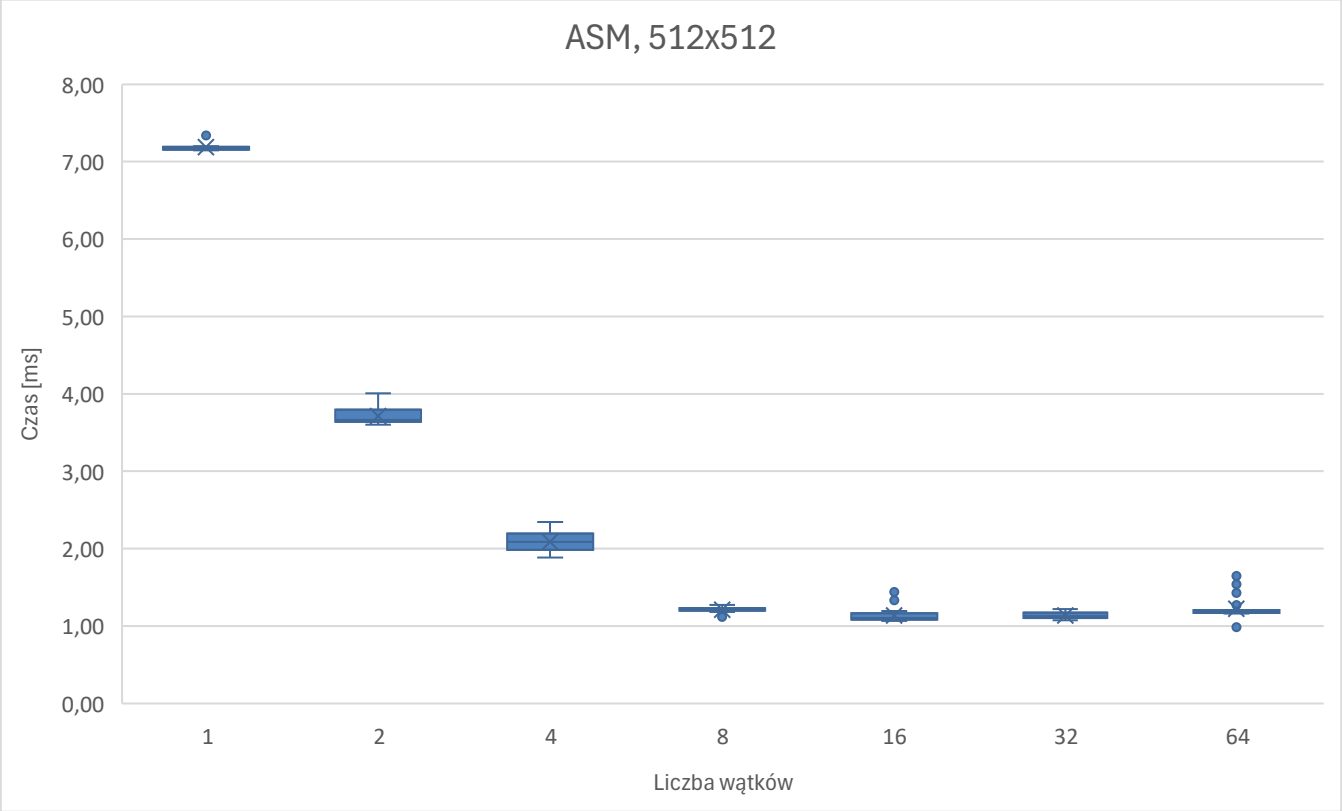
Rysunek 1: Zdjęcie programu przed dodaniem szumu

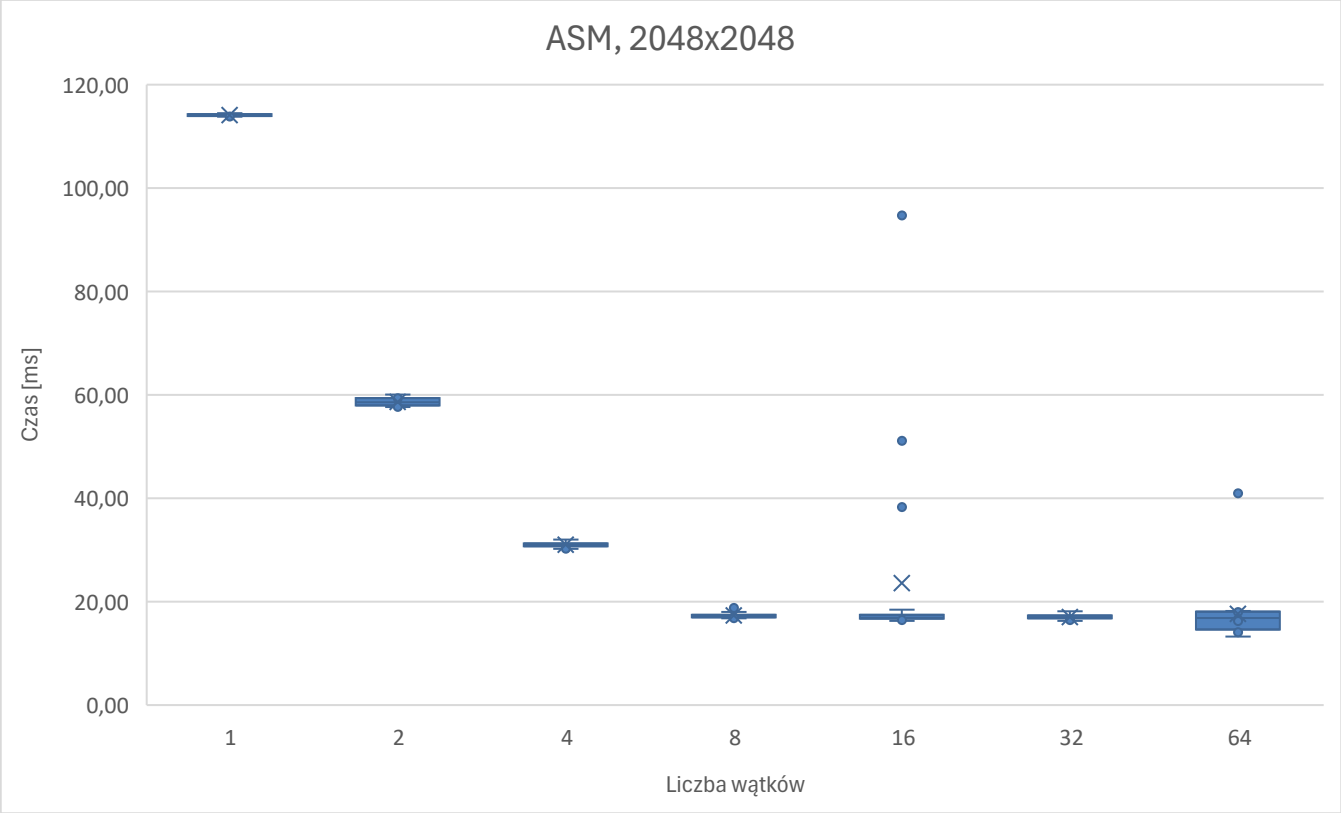


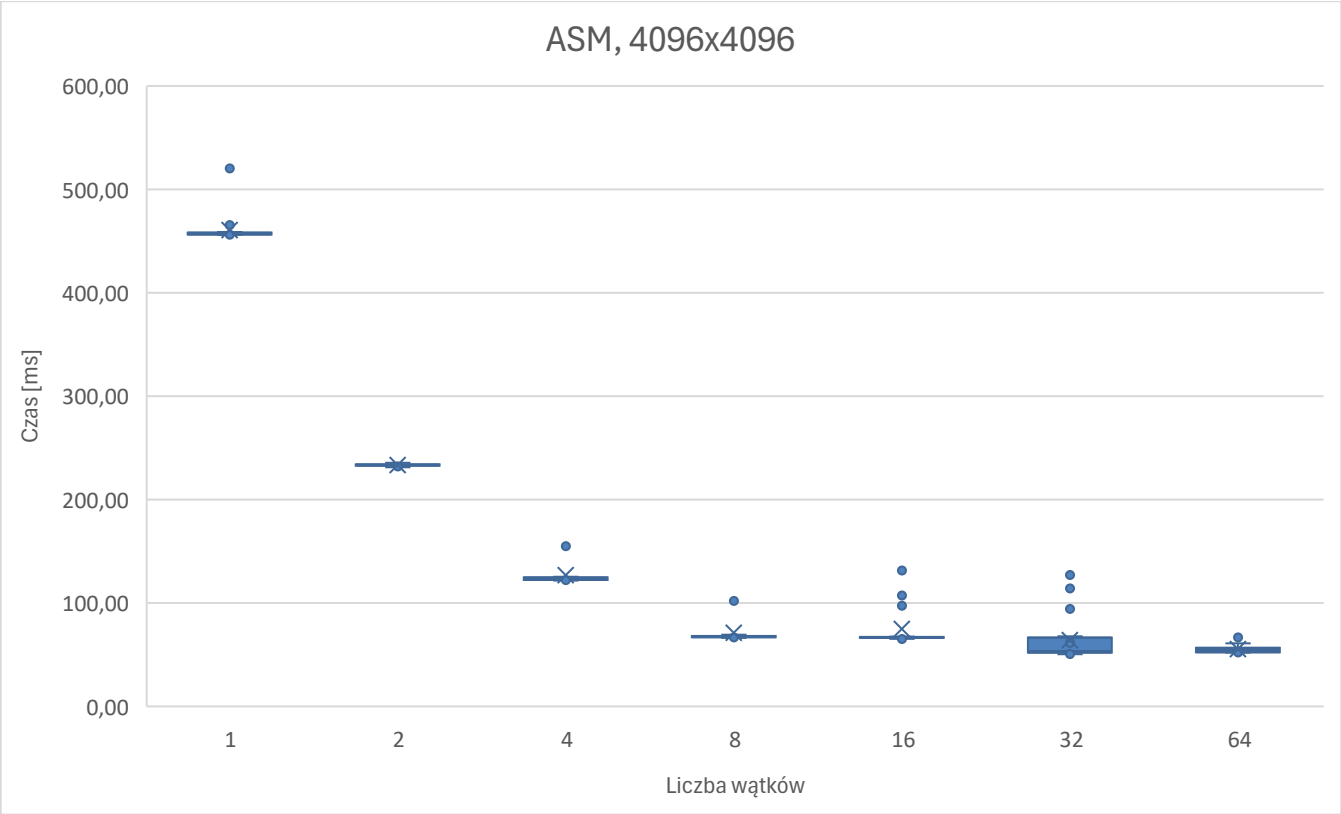
Rysunek 2: Zdjęcie programu po dodaniu szumu

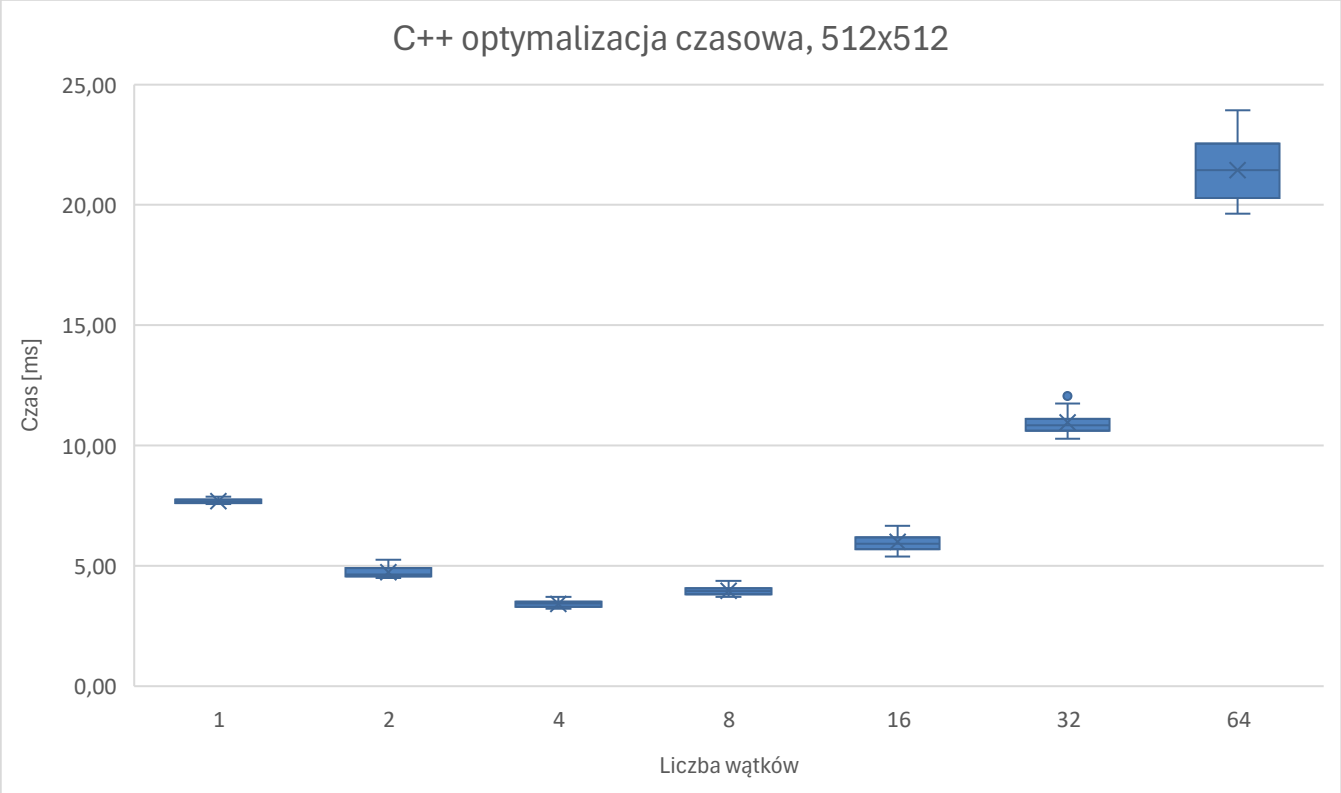
Raport szybkości działania

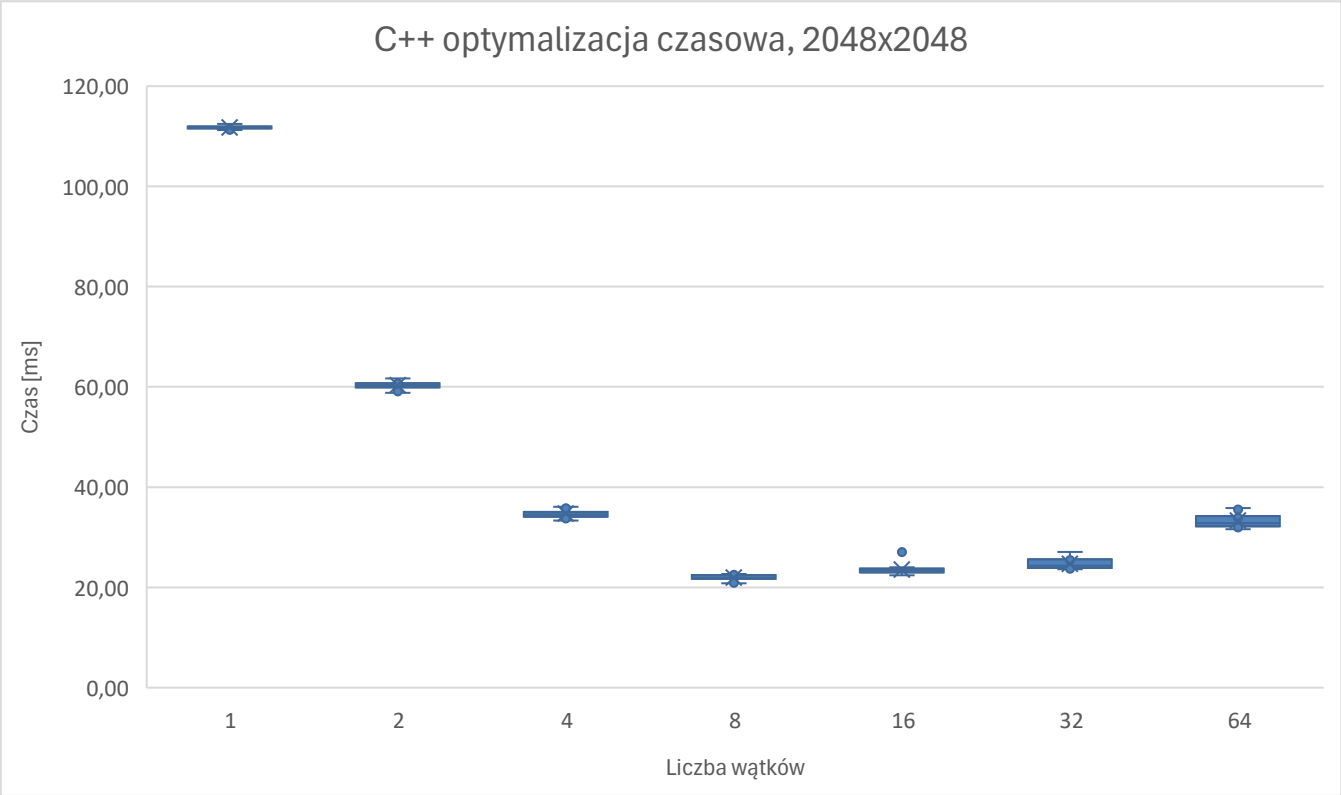
Dla każdego wątku czas został zmierzony 20 razy.

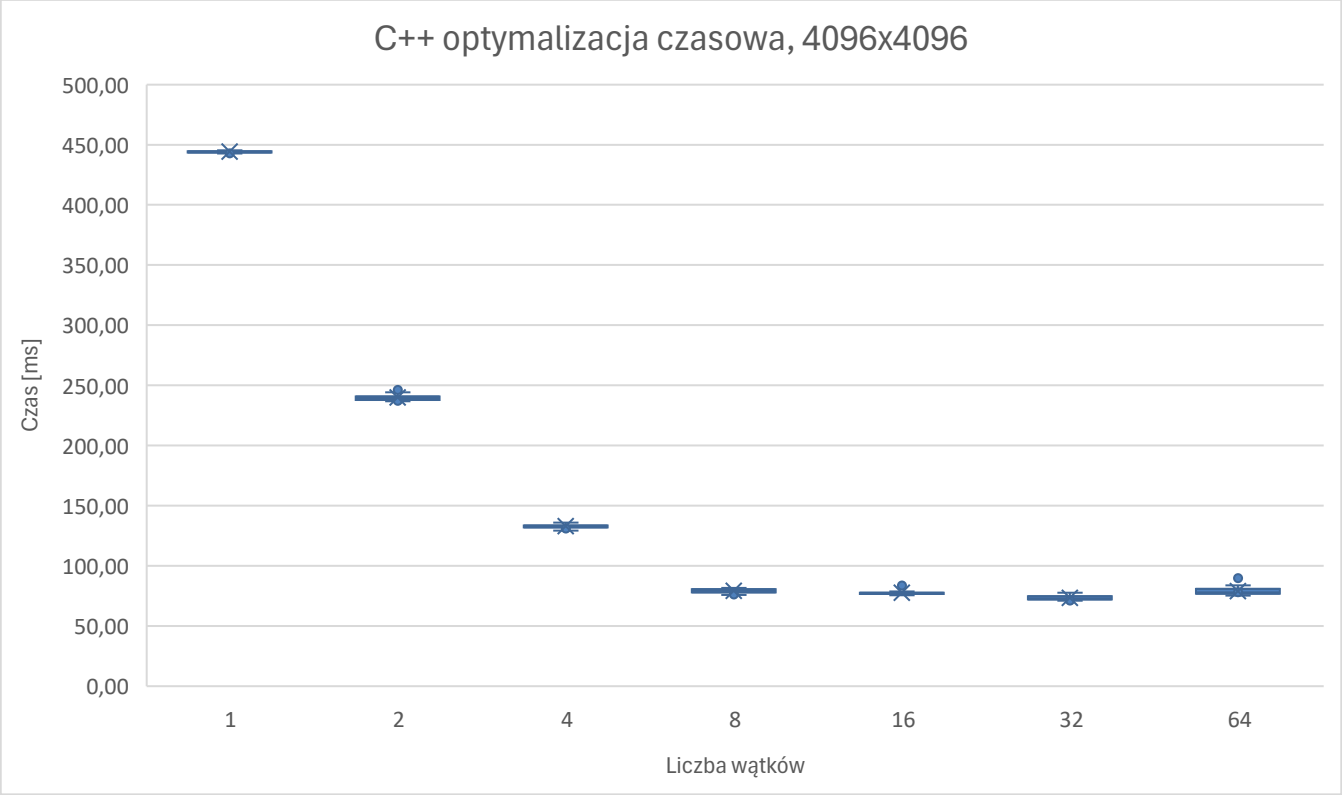


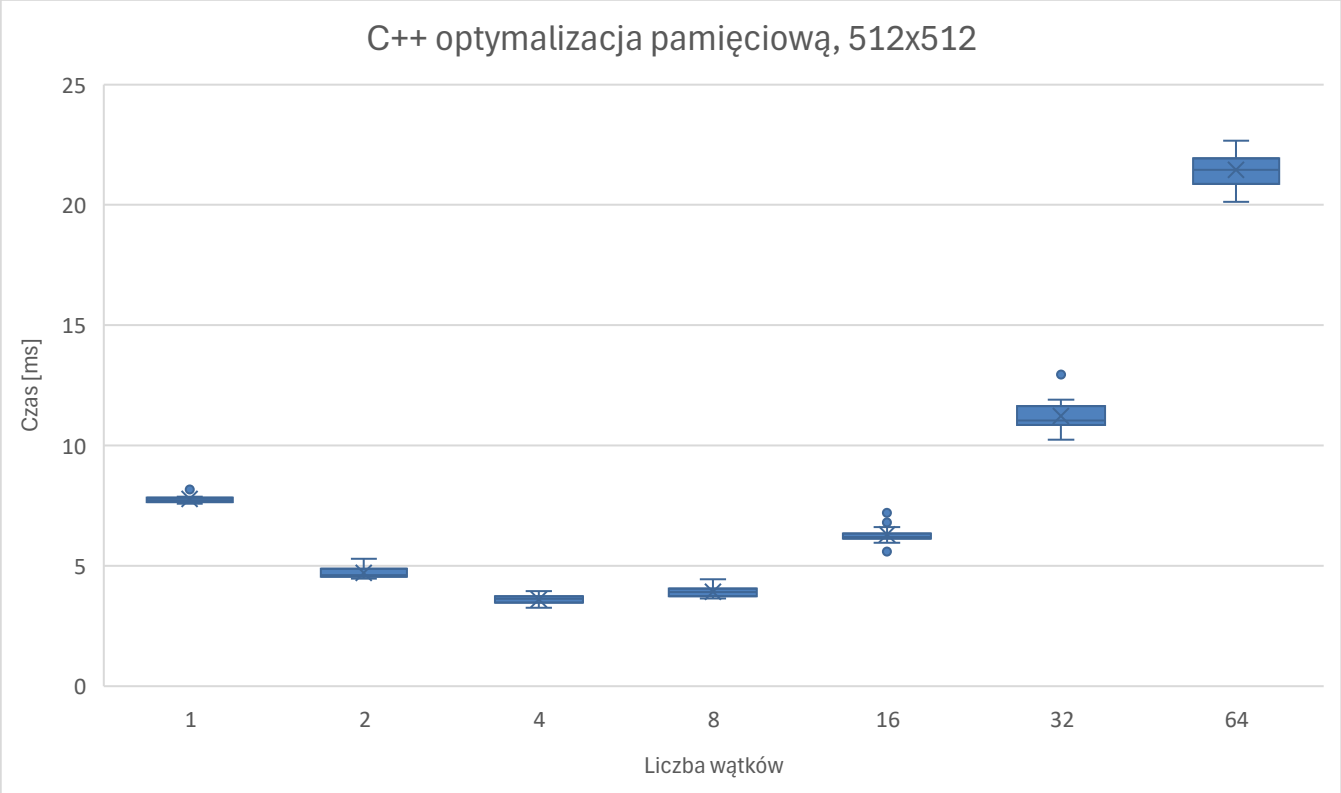

ASM, 2048x2048
120,00
100,00
80,00
60,00
40,00
20,00
0,00
Czas [ms]
1
2
4
8
16
32
64
Liczba wątków

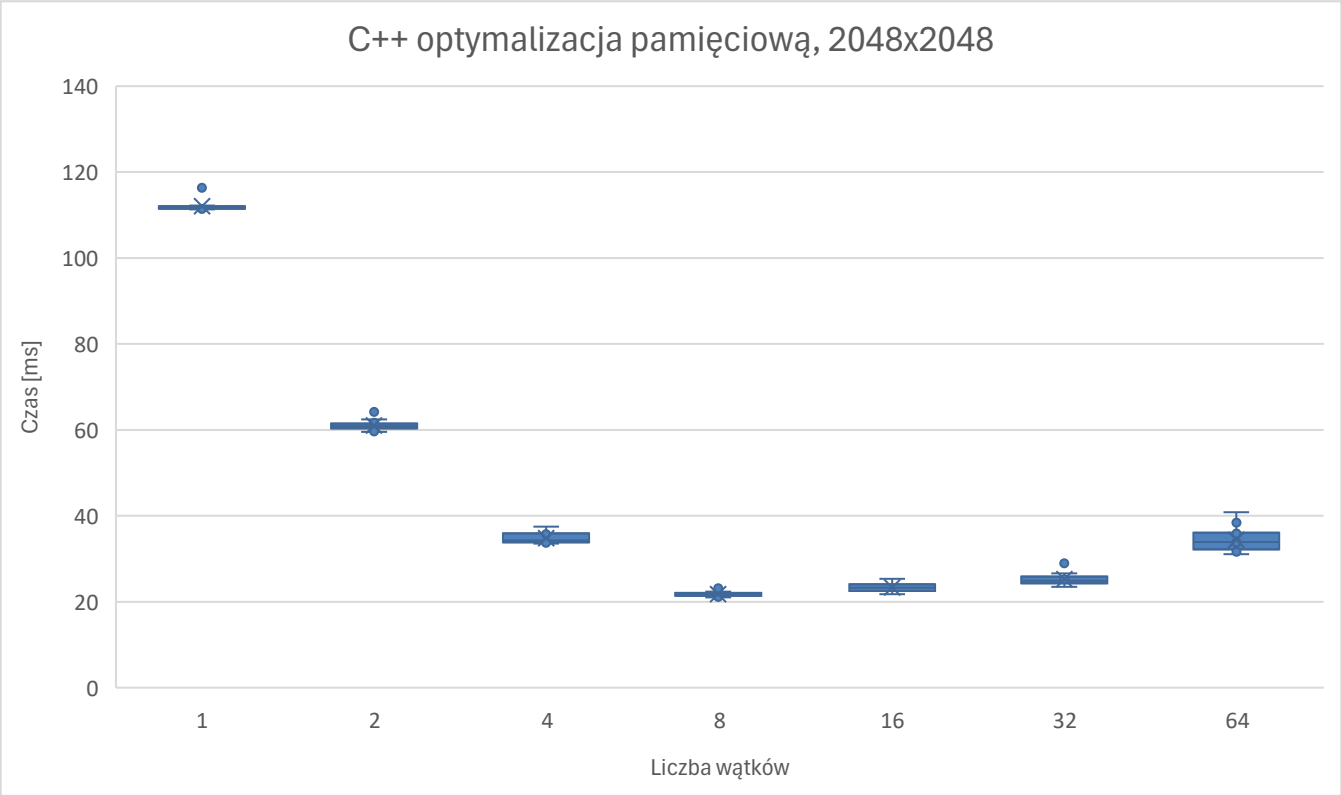

ASM, 4096x4096
600,00
500,00
400,00
300,00
200,00
100,00
0,00
Czas [ms]
1
2
4
8
16
32
64
Liczba wątków

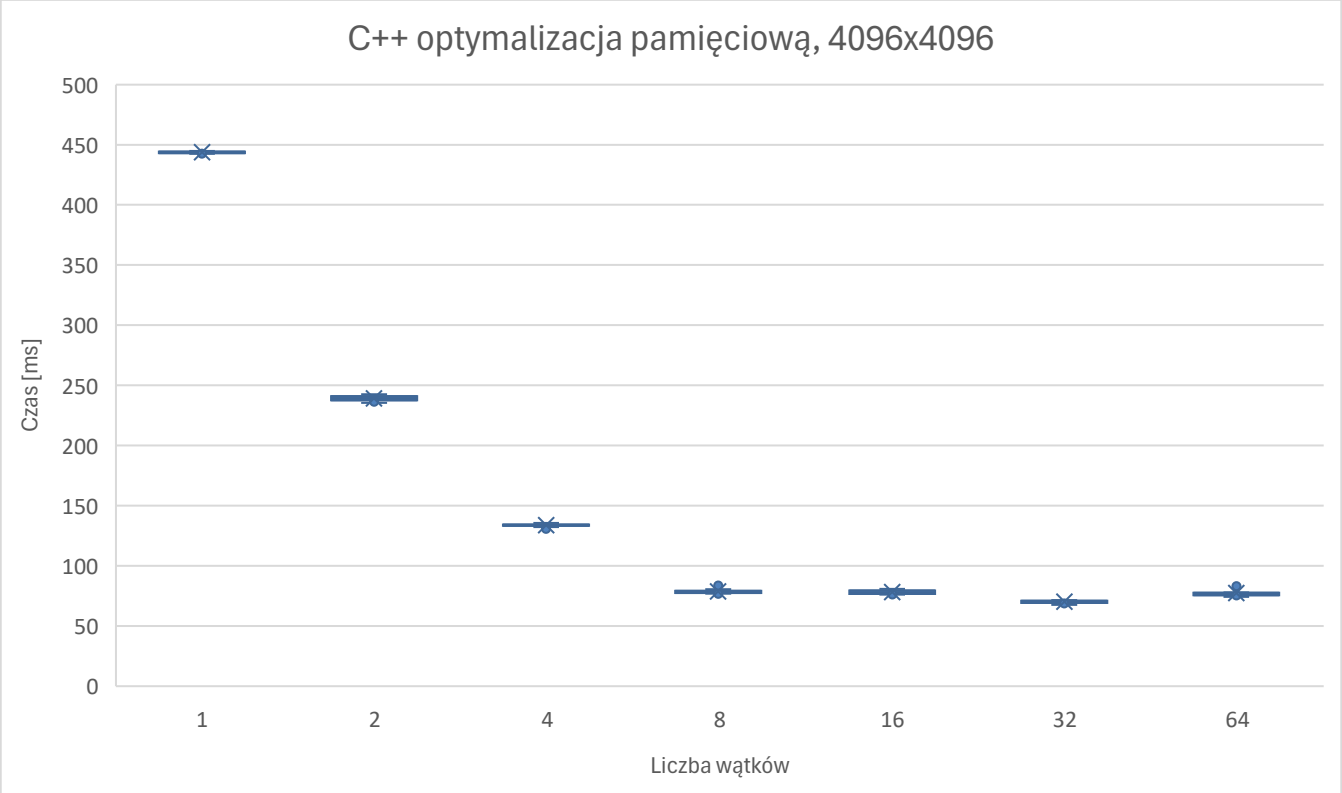

C++ optymalizacja czasowa, 512x512
25,00
20,00
15,00
10,00
5,00
0,00
Czas [ms]
1
2
4
8
16
32
64
Liczba wątków


C++ optymalizacja czasowa, 2048x2048
120,00
100,00
80,00
60,00
40,00
20,00
0,00
Czas [ms]
1
2
4
8
16
32
64
Liczba wątków


C++ optymalizacja czasowa, 4096x4096
500,00
450,00
400,00
350,00
300,00
250,00
200,00
150,00
100,00
50,00
0,00
Czas [ms]
1
2
4
8
16
32
64
Liczba wątków


C++ optymalizacja pamięciową, 512x512
Czas [ms]
25
20
15
10
5
0
1
2
4
8
16
32
64
Liczba wątków


C++ optymalizacja pamięciową, 2048x2048
Czas [ms]
140
120
100
80
60
40
20
0
1
2
4
8
16
32
64
Liczba wątków



Opis testowania i uruchamiania programu

Program uruchomiono i przetestowano na środowisku wyposażonym w procesor Intel core i5-12400f oraz 64-bitowy system operacyjny Windows. Procesor ten wspiera zaawansowane instrukcje wektorowe SIMD (m.in. vpbroadcastw czy vpunpcklqdq).

Podczas testów funkcjonalnych zweryfikowano poprawność działania algorytmu poprzez wizualną ocenę obrazu przy zmiennych parametrach siły szumu oraz potwierdzono możliwość poprawnego zapisu zdjęć. Wydajność aplikacji zmierzono przy użyciu klasy Stopwatch, porównując czas przetwarzania dla implementacji w C++ względem wersji asemblerowej.

Wnioski

Implementacja asemblerowa jest w większości przypadków szybsza od implementacji c++. Nie zauważono tutaj znacząco szybszego wykonywania algorytmu asemblerowego względem c++ dlatego, że wykorzystana została transformacja Boxa-Mullera, która to korzysta z logarytmów oraz funkcji trygonometrycznych które nie mają swojej wersji wektorowej w asemblerze. Dlatego też transformacja ta była wąskim gardłem w programie i asembler był szybszy tylko w trakcie dodawania szumu do pikseli. Aby zauważyć znaczące przyśpieszenie asemblera względem c++ należałoby wykorzystać rozkład Irwina-Halla zamiast transformacji Boxa-Mullera. Rozkład ten nie wykorzystuje logarytmów czy funkcji trygonometrycznych tylko sumę losowych liczb o rozkładzie jednostajnym. Na wykresach można zauważyć, że wydajność implementacji c++ znacząco spada przy większej ilości wątków podczas gdy wydajność asemblera zmniejsza się nieznacznie. Może to być spowodowane tym, że czas zaoszczędzony na przetwarzaniu obrazu przez kilka wątków jest mniejszy od czasu tworzenia samych wątków przy użyciu biblioteki thread.